

Chapter 16

CS2 course package using Python - Webster

Xiaoyuan Suo¹

¹Webster University

CS2 Python Course Materials Used at Webster University

Abstract

[This chapter presents a modular approach for integrating parallel and distributed computing (PDC) concepts into a second programming course (CS2) using Python. The modules are organized around three progressively advanced activities. The first module introduces basic parallel programming concepts through Python threading and multiprocessing. The second module applies multiprocessing to image processing tasks using OpenCV, allowing students to parallelize grayscale conversion, image filtering, and image segmentation operations. The third module introduces basic performance engineering concepts including timing methodologies, speedup, efficiency, Amdahl's Law, weak versus strong scaling, memory-bound versus compute-bound workloads, and NUMA-related performance effects. Students experimentally evaluate runtime behavior using multicore systems and visualize performance trends using plots and benchmarking tools.]

Descriptor Elements

Programming Language Employed: [Python]

Relevant core courses: [CS2, Data Structures, Advanced Python Programming]

Pre-requisites: [CS1, at Webster University, we require students to have sufficient knowledge in C++]

Relevant PDC topics:

PDC Concept	Blooms Level
Introduction to Parallel Computing	K
Threads and Multiprocessing	C
Race Conditions and Synchronization	A
Task Decomposition and Data Partitioning	A
Performance Measurement and Benchmarking	A
Speedup and Efficiency	C
Strong vs. Weak Scaling	C
Parallel Overhead	C
Memory-bound vs. Compute-bound Workloads	C
NUMA Awareness and Memory Locality	K

Learning outcomes:

1. Explain the differences between sequential, parallel, and distributed computing.

2. Develop basic multithreaded and multiprocessing programs in Python.
3. Identify and resolve race conditions using synchronization mechanisms such as locks and RLocks.
4. Apply task decomposition techniques to parallelize computational workloads.
5. Analyze runtime, speedup, and efficiency of parallel programs using empirical measurements.
6. Interpret scalability behavior using Amdahl's Law and weak/strong scaling concepts.
7. Implement parallel image processing applications using Python multiprocessing and OpenCV.
8. Evaluate the effects of parallel overhead, memory bottlenecks, and workload granularity on performance.

Context for use: |The materials were developed for a Python-based CS2 course at Webster University, which is a primarily undergraduate institution (PUI). The course typically enrolls 10-25 undergraduate computer science students with limited prior exposure to parallel computing or computer architecture concepts but with knowledge in C++. The modules are designed to be lightweight and approachable for students without advanced mathematical backgrounds. Python was selected because of its accessibility and extensive ecosystem support for multiprocessing, visualization, and image processing libraries such as OpenCV and NumPy. The activities can be conducted on standard desktop or laptop computers without requiring access to specialized HPC hardware, although optional extensions may leverage multicore servers or institutional clusters.|

16.1 Introduction

Python was selected for this course because students enter with prior programming experience in C++, allowing them to focus on computational thinking and PDC concepts rather than low-level syntax. Python also provides a gentle transition into parallel and distributed computing through its simple syntax, rich scientific libraries, and accessibility for STEM students outside traditional computer science backgrounds. In addition, Python is widely used in data science, artificial intelligence, and research computing, which makes it attractive to undergraduate students (not only computer science majors).

Despite these advantages, Python is not always ideal for teaching PDC compared to languages such as C++ or Fortran. Python introduces limitations related to the Global Interpreter Lock (GIL), lower execution performance, and reduced exposure to low-level memory management and hardware optimization. As a result, some advanced performance engineering topics must be simplified or demonstrated conceptually rather than implemented at full scale. Nevertheless, Python enables rapid experimentation and visualization, which is particularly valuable in an introductory undergraduate setting.

16.2 How CS2 was changed and PDC topics integrated

I started integrating PDC topics into this Python course since 2020 through small iterative modifications each year. Introductory PDC concepts are presented in Weeks 4-5, after students have become familiar with core Python data structures such as lists and dictionaries. Parallel image processing activities are introduced around Week 8 after students gain experience with files and binary data. Basic performance engineering concepts are introduced near the end of the semester after the students have developed stronger programming and debugging skills.

To make room for the new material, several traditional topics were reduced or streamlined. Less time is spent on low-level image processing details and introductory SQLite programming because these topics are covered more thoroughly in other courses within the curriculum. The revised structure emphasizes computational thinking, scalability, and experimental analysis while preserving the core programming foundations of the original course.

16.2.1 Integrated Syllabi (Pre and Post)

Table 16.1 summarizes the major differences between the original course structure and the revised syllabus after integrating PDC concepts and activities.

16.2.2 Highlights

Three PDC-related topics are included in the teaching materials.

- **Basics of Parallel and Distributed Computing:** The introductory module introduces students to the foundations of parallel and distributed computing using accessible real-world examples and simplified Python demonstrations. Topics include task decomposition, threads, synchronization, race conditions, locks, deadlocks, multiprocessing,

Table 16.1: Comparison of course organization before and after PDC integration.

Topic Area	Original Course	Revised Course with PDC
Python fundamentals	Full coverage	Full coverage
Lists and dictionaries	Full coverage	Full coverage with task decomposition examples
File processing	Traditional file I/O	File I/O with parallel image processing examples
Image processing	Detailed sequential coverage (such as OpenCV and Pillow),	Reduced depth; integrated parallel image processing activities such as image blurring and black-and-white conversion.
SQLite / Python database programming	Multiple weeks	Reduced coverage
Introduction to PDC	Not included	Added in Weeks 4–5
Parallel image processing	Not included	Added in Weeks 8–9
Performance engineering	Not included	Added near end of semester
Experimental benchmarking	Minimal emphasis	Added through timing and scalability exercises

process pools, synchronous versus asynchronous execution, and distributed computing concepts. Students are also introduced to the motivations behind PDC, including scalability, computational efficiency, and solving larger real-world problems.

- **Parallel Image Processing and Vectorization:** I included teaching material that introduces parallel image processing through beginner-friendly activities such as image blurring, grayscale conversion, and image decomposition. These activities allow students to visualize how large problems can be partitioned into smaller independent tasks. In addition to parallelism, the course introduces the concept of vectorization and explains why modern computing systems rely heavily on vectorized operations for performance. Real-world analogies, such as multiple chefs preparing meals simultaneously (parallelism) versus a single chef using specialized tools to process many ingredients at once (vectorization), help students intuitively distinguish between the two concepts (see Table 16.2).
- **Introduction to Performance Engineering:** The course also introduces basic performance engineering concepts, which are rarely emphasized in introductory undergraduate programming courses. Students learn why performance matters in modern computing and are exposed to topics such as runtime measurement, benchmarking, scalability, speedup, efficiency, and the distinction between compute-bound and

memory-bound applications. Rather than focusing solely on correctness, I encouraged students to experimentally evaluate and interpret program performance using timing and comparative analysis.

Table 16.2: Comparison between parallel programming and vectorization.

Feature	Parallel Programming (Multiprocessing)	Vectorization (SIMD)
Level	Multi-core execution (high-level parallelism)	Single-core data-level optimization (low-level SIMD operations)
Real-world analogy	Hiring four chefs to cook four different pizzas simultaneously	Using a specialized machine to cook four pizzas at once, or a large stamp-like cutter to slice many pepperoni pieces simultaneously
Common Python tools	multiprocessing, joblib	NumPy, Pandas
Hardware focus	Multiple CPU cores	SIMD registers (<i>Single Instruction, Multiple Data</i>)

16.2.3 Challenges

Limitations of Using Python for PDC One challenge of teaching PDC in Python is that the language is not designed primarily for high-performance computing. Compared to C++ or Fortran, Python provides less direct control over memory management, thread scheduling, and hardware-level optimization. The Global Interpreter Lock (GIL) also limits true multithreaded execution in many situations, requiring instructors to simplify some discussions or rely on higher-level libraries and multiprocessing approaches.

The Lack of an Appropriate Textbook Another challenge is the lack of beginner-friendly textbooks that integrate Python programming with introductory PDC concepts. Most available PDC materials assume stronger mathematical or systems programming backgrounds and are commonly written using C/C++ with MPI or OpenMP. Considerable effort was therefore required to develop custom teaching materials, animations, and assignments suitable for early undergraduate students.

Generative AI As mentioned in the previous chapter, modern generative AI tools create challenges in assignment design and assessment. Many introductory programming exercises can now be answered quickly using AI-assisted coding tools. As a result, assignments increasingly emphasize experimentation, interpretation of results, performance analysis, and reflection rather than code implementation alone.

16.3 New Unplugged Activities

Unplugged activities used in my class can be found in the previous chapter 15.2.2

In addition, I developed a distributed computing simulation activity in which students role-play as users (3), a dispatcher node (1), and worker processors (other students). Users submit some simple computational tasks to a central dispatcher, which distributes the work among available processors and collects the results. Worker processors may enter states such as *idle*, *running*, *waiting*, or *failed*, allowing students to visualize concepts including task scheduling, communication overhead, load balancing, synchronization, scalability, and fault tolerance in distributed systems.

The activity demonstrates several key PDC concepts:

PDC Concept	How the Activity Demonstrates It
Task distribution	Dispatcher assigns jobs to workers
Load balancing	Some workers may become overloaded while others remain idle
Communication overhead	Time spent passing messages between users, dispatcher, and workers
Fault tolerance	A worker may “fail” before completing a task
Scalability	More workers can process more jobs simultaneously
Synchronization	Results must return before final answers are assembled
Resource management	Dispatcher decides which worker receives tasks
Distributed systems architecture	Multiple independent processors collaborate on one overall problem

16.4 New Plugged-in Activities

16.4.1 Zombie Attack

A description of the Zombie Attack assignment can be found in the previous chapter ??.

A Python version of the same assignment is available in my GitHub Repository.

Problem Description, Prerequisites, and Learning Outcomes

Algorithms

Implementation Details

- Instructional Material (Lectures, Videos, Assignments, Tests)

- **How-To Guides**

Experience and Teaching Tips

Data and Analysis

16.5 Course-wide Pre and Post Course Surveys

Survey results from COSC 2110 indicate measurable gains in students' understanding of PDC-related concepts and computational problem solving. Before the course, students reported an average self-rating of 2.75/4.0 for parallel processing experience and 2.50/4.0 for distributed computing experience, indicating only moderate familiarity with these topics. Interest in learning about parallel and distributed computing was already relatively high at 5.33/7.0, while belief in the importance of PDC averaged 5.89/7.0. Students' self-reported understanding of parallel and distributed computing averaged 4.22/7.0 prior to instruction.

Post-course responses demonstrated stronger problem-solving and computational reasoning perspectives. Students reported an average rating of 5.19/7.0 on their ability to critically assess and select algorithms for specific applications. Problem decomposition skills averaged 2.54/4.0, while parallel and distributed computing experience ratings remained above the moderate-experience range at 2.58/4.0 and 2.60/4.0 respectively. Open-ended responses frequently referenced optimization, decomposition, benchmarking, scalability, and performance-aware thinking as major learning outcomes, suggesting that students increasingly approached programming problems from both correctness and efficiency perspectives.

16.6 Other Activities and Ideas

CS2 vs. CS1 My experiences from multiple years of this course offering suggest that these PDC topics are difficult to integrate effectively into an introductory Python programming course. Many concepts, including task decomposition, scalability, benchmarking, and multiprocessing, require students to already possess sufficient programming maturity and debugging skills. As a result, the above-mentioned material is better suited for a CS2-level course, where students already have prior exposure to programming fundamentals and basic data structures.

HPC Topics Future versions of the course may incorporate additional HPC-related activities, including introductory cluster usage, remote job submission, and simple distributed computing demonstrations using shared computing resources. Introducing students to HPC systems early may help bridge the gap between traditional programming instruction and modern computational research practices.

Performance Engineering Performance engineering was one of the most valuable yet challenging topics to integrate into the course. Because of time limitations and student background constraints, my course focused primarily on introductory concepts such as runtime measurement, benchmarking, speedup, efficiency, multiprocessing overhead, and distin-

guishing between compute-bound and memory-bound applications. More advanced topics, including cache optimization, NUMA effects, compiler optimization, vector register utilization, and low-level hardware profiling, were intentionally left out because most students had not yet completed computer architecture or algorithms.

Another important observation is that students are highly motivated by modern and visually engaging applications. Activities involving image processing, animations, AI-related examples, and experimental benchmarking consistently generated stronger engagement compared to purely theoretical discussions. Future work may include additional interactive visualizations, lightweight HPC modules, and expanded performance engineering case studies that connect directly to real-world applications.

16.7 Conclusion

Integrating PDC concepts into a CS2-level Python course provides students with early exposure to modern computational thinking and performance-aware programming. Although Python presents certain limitations for teaching advanced HPC concepts, its accessibility and rich ecosystem make it an effective platform for introducing parallelism, vectorization, scalability, and experimental performance analysis.

Bibliography

Appendix

|You must include these four specific subsections (in addition to anything else):|

Github Link to Instructional Material

<https://github.com/xiaoyuansuo51-webster/Teaching-PDC-Python->

Detailed Pre and Post Syllabus

Syllabus can be found: Course Syllabus

The course follows a traditional Python programming sequence during the first half of the semester. Weeks 1–3 focus on Python syntax, control structures, functions, and file processing. During this period, students develop sufficient programming fluency to begin reasoning about larger computational tasks and data organization.

PDC concepts are intentionally introduced during Weeks 4–5, immediately after students complete instruction on lists, dictionaries, and sets. At this point, students are capable of understanding how data collections can be partitioned and processed independently. Rather than introducing formal parallel algorithms immediately, the course first emphasizes conceptual ideas such as task decomposition, workload distribution, multiprocessing, synchronization, race conditions, and deadlocks. These topics are reinforced using unplugged activities, classroom demonstrations, and simple Python multiprocessing examples. Introducing PDC topics too early in the semester was found to be ineffective because students were still struggling with basic Python syntax and debugging.

Parallel image processing activities are introduced around Week 8 after students gain experience with recursion and file processing. This placement was intentional because students are more comfortable manipulating binary data and working with larger programs by this stage of the semester. Image processing also provides a highly visual and naturally parallel problem domain. Students experiment with operations such as grayscale conversion, image decomposition, and filtering while discussing concepts including data partitioning, vectorization, and SIMD-style computation. The visual nature of image manipulation allows students to immediately observe the effects of parallel decomposition strategies.

Basic performance engineering concepts are introduced near the end of the semester after students develop stronger programming, debugging, and program analysis skills. The final PDC-related activities focus on benchmarking, runtime measurement, speedup, efficiency, scalability, and multiprocessing overhead. Positioning performance engineering later in the semester allows students to concentrate on interpreting experimental results rather than struggling with implementation details. Students are required to document runtime behavior, compare sequential and parallel implementations, and analyze cases where additional parallelism may not improve performance due to overhead costs.

Organization of Material

Each module contains lecture slides, assignments, instructor notes, and supporting documents designed to help instructors integrate the material into a CS2-level Python course

with minimal preparation.

Organization of Material

- **Module 1: Basics of PDC**

- **Files Included:**

- * Python parallel.pdf
 - * Assignment12
 - * Read me.pdf

- **Purpose and Content:**

Introduces the foundations of parallel and distributed computing using Python examples and real-world analogies. Topics include multiprocessing, synchronization, locks, race conditions, deadlocks, and task decomposition. The programming assignment asks students to implement a parallel summation program and experimentally document performance behavior. Since such assignments can be easily generated using AI tools, students are required to complete a substantial portion of the implementation during supervised class activities and later document runtime behavior and observations independently. Instructor notes provide suggestions for demonstrations, pacing, and classroom discussions.

- **Module 2: Parallel Image Processing and Vectorization**

- **Files Included:**

- * parallel image.pdf
 - * Assignment
 - * read me.pdf

- **Purpose and Content:**

Introduces parallel image processing through visual examples such as image blurring, grayscale conversion, and image decomposition. This module also introduces vectorization and discusses why modern computing systems rely heavily on vectorized operations for performance optimization. Students compare traditional multiprocessing approaches with vectorized NumPy operations to better understand the distinction between parallel execution and SIMD-style computation. Instructor notes highlight conceptual discussions and implementation details.

- **Module 3: Performance Engineering**

- **Files Included:**

- * Basic Performance Engineering Python.pdf
- * Performance engineering notes.pdf
- * read me.pdf

– **Purpose and Content:**

Introduces introductory performance engineering concepts including benchmarking, runtime measurement, scalability, speedup, efficiency, multiprocessing overhead, and experimental analysis. Because suitable undergraduate-level textbooks covering these topics in Python were difficult to identify, supplemental notes were developed to provide concise explanations and examples. Assignments emphasize documenting experimental results and interpreting performance behavior rather than focusing solely on program correctness, which also helps reduce overreliance on AI-generated solutions. Instructor notes include implementation suggestions and classroom discussion points.